University of Dhaka

Department of Computer Science and Engineering

CSE 2206 – Microcontroller and Embedded System

Lab Assignment 3

BME280 Environmental Sensor Interface

Part A: SPI Communication & Part B: I2C Communication

Bare-Metal (Register-Level) & HAL / STM32CubeIDE

Submitted to

Dr. Mosaddek Hossain Kamal (Tushar)

Professor

Dept. of CSE, DU

Submitted by

MD.Jahidul Islam Sarker

Roll: AE - 01

Registration No.: 2023015944

A6.5 Test A5 — HAL vs Bare-Metal Comparison

Flashed bare-metal, recorded three readings. Flashed HAL version, recorded three readings.

Table 1: A6.5 Test A5 — HAL vs Bare-Metal Comparison

Reading	BM Temp (°C)	HAL Temp (°C)	Diff	Pass (≤0.1°C)?
1	34.92	34.93	0.01	Yes
2	34.93	34.94	0.01	Yes
3	34.92	34.95	0.03	Yes

Measurement time: 30 May 2026 at 9:08 PM

B6.5 Test B5 — HAL vs Bare-Metal (I2C)

Flashed bare-metal, recorded three readings. Flashed HAL version, recorded three readings.

Table 2: B6.5 Test B5 — HAL vs Bare-Metal (I2C)

Reading	BM Temp (°C)	HAL Temp (°C)	Diff	Pass (≤0.1°C)?
1	34.85	34.82	0.03	Yes
2	34.85	34.81	0.04	Yes
3	34.84	34.81	0.03	Yes

Measurement time: 30 May 2026 at 9:20 PM

Final Comparison: SPI vs I2C

The two parts of the assignment gave almost the same sensor readings. SPI felt a bit more direct because it used a separate chip-select line, while I2C kept the wiring simpler with only two lines. The table below shows the comparison using my own measurements.

Table 3: Final Comparison Between SPI and I2C for the BME280

Aspect	**Part A: SPI**	**Part B: I2C**
Wires to BME280	4 (MOSI, MISO, SCK, CS)	2 (SDA, SCL)
STM32 Peripheral	SPI2	I2C1
Bare-Metal Config	6 register writes	5 register writes + open-drain setup
HAL API	HAL_SPI_Transmit/Receive	HAL_I2C_Mem_Read/Write
Max Bus Speed	45 Mbps (fPCLK/2)	400 kHz (Fast mode)
Pull-up Resistors	Not required	Required (4.7 kΩ to 3.3 V)
CS Pin Required	Yes (PB9)	No – address-based
Multiple Slaves	One CS per slave	Address-based sharing
Your Temp (°C)	34.92	34.85
Your Pressure (hPa)	971.08	971.07
Readings Match?		Yes (within 0.07 °C)

Discussion Questions

1. When would you choose SPI over I2C in a real product?

I will choose SPI over I2C when

i) Speed is Critical : SPI runs up to 45 Mbps on STM32F446RE versus I2C's maximum 400 kHz. It is the preferred choice for high-frequency sensor polling, display driving, or real-time data logging where throughput matters.

ii) Full-Duplex Mode is Needed : SPI has separate MOSI and MISO lines, allowing simultaneous send and receive on the same clock cycle. I2C shares a single SDA line making it half-duplex only.

iii) Noise-Resilient Communication is Needed : SPI uses dedicated point-to-point lines with no shared bus, making it more robust in electrically noisy environments like motor controllers or industrial systems.

iv) Number of Peripherals is Limited : When only 2–3 slaves are present and enough GPIO pins are available, the CS-per-slave requirement of SPI is not a burden and its speed advantage can be fully utilized.

and I2C over SPI when

i)Pin Count is Tight : I2C needs only 2 wires (SDA, SCL) for any number of slaves, versus SPI needing 4 wires plus one extra CS pin per additional slave. This is critical in wearables or pin-constrained MCUs.

ii) Many Slow Sensors Share a Bus :Environmental sensors like BME280 rarely need speeds beyond 400 kHz. I2C allows multiple such sensors on the same two wires using unique 7-bit addresses, saving both pins and PCB routing.

iii) Board Simplicity Matters : I2C eliminates per-device CS routing and uses address-based selection, reducing PCB complexity and firmware GPIO management significantly.

2. The BME280 outputs identical sensor data regardless of whether SPI or I2C is used. Why, then, is the firmware compensation code the same in both parts?

The compensation code remains the same because it depends on the BME280 sensor model, not on the bus protocol. SPI and I2C only control how the raw register bytes are transferred from the sensor to the microcontroller. Once those bytes are received correctly, the data format is exactly the same in both cases.

The firmware then applies the same calibration constants and the same compensation equations specified in the datasheet. This is why temperature, pressure values come out nearly identical in both parts. In other words, the communication code differs between Part A and Part B, but the compensation logic remains completely unchanged. In short, the sensor hardware does not change, only the communication wrapper around it does.